

CSE 3204
Formal Language, Automata and Computability



Lecture 19
Decision Properties of CFL and Turing Machine

Testing Emptiness of CFL

Let $G = (V, T, P, S)$ be a grammar. To compute the generating symbols of G , we perform the following induction.

BASIS: Every symbol of T is obviously generating; it generates itself.

INDUCTION: Suppose there is a production $A \rightarrow \alpha$, and every symbol of α is already known to be generating. Then A is generating. Note that this rule includes the case where $\alpha = \epsilon$; all variables that have ϵ as a production body are surely generating.

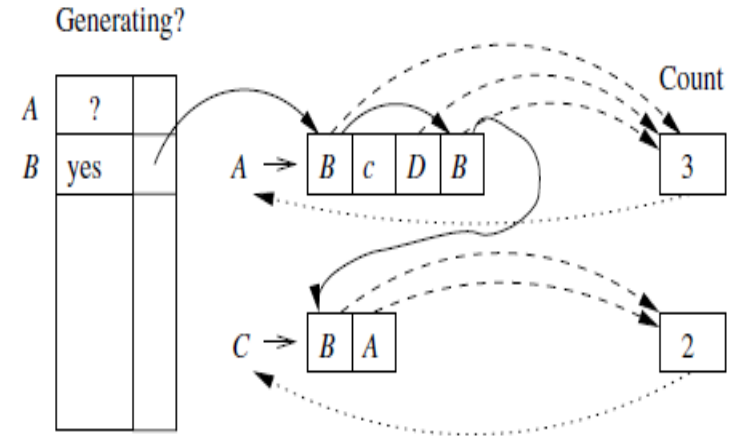


Figure 7.11: Data structure for the linear-time emptiness test

The execution time is on the order of $O(n)$

- ✓ Test whether S is generating in CFG G .
- ✓ The Naïve approach requires $O(n^2)$, where n is the size of the CFG, i.e., order of variable is n .

Testing Membership of a String in a CFL

- Finding whether w in CFL has a running time that is exponential to the length of w .
- The *CYK* algorithm construct a table to find whether w in CFL in $O(n^3)$ time, n is the length of w .
- This algorithm works on CNF $G = (V, T, P, S)$ for a language L . The input is a string $w = a_1 a_2 \dots a_n$ in T^*

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12}	X_{23}	X_{34}	X_{45}	
X_{11}	X_{22}	X_{33}	X_{44}	X_{55}
a_1	a_2	a_3	a_4	a_5

$a_1 a_2 \dots a_n$, which we have supposed has length 5. The table entry X_{ij} is the set of variables A such that $A \xRightarrow{*} a_i a_{i+1} \dots a_j$. Note in particular, that we are interested in whether S is in the set X_{1n} , because that is the same as saying $S \xRightarrow{*} w$, i.e., w is in L .

- ✓ It takes $O(n)$ to compute any entry.
- ✓ The table contains $n(n+1)/2$ entries, resulting in $O(n^3)$ time.

Figure 7.12: The table constructed by the CYK algorithm

Construction of X_{ij}

BASIS: We compute the first row as follows. Since the string beginning and ending at position i is just the terminal a_i , and the grammar is in CNF, the only way to derive the string a_i is to use a production of the form $A \rightarrow a_i$. Thus, X_{ii} is the set of variables A such that $A \rightarrow a_i$ is a production of G .

INDUCTION: Suppose we want to compute X_{ij} , which is in row $j - i + 1$, and we have computed all the X 's in the rows below. That is, we know about all the case $i = j$ is the basis), we know that any derivation $A \xRightarrow{*} a_i a_{i+1} \cdots a_j$ must start out with some step $A \Rightarrow BC$. Then, B derives some prefix of $a_i a_{i+1} \cdots a_j$, say $B \xRightarrow{*} a_i a_{i+1} \cdots a_k$, for some $k < j$. Also, C must then derive the remainder of $a_i a_{i+1} \cdots a_j$, that is, $C \xRightarrow{*} a_{k+1} a_{k+2} \cdots a_j$.

We conclude that in order for A to be in X_{ij} , we must find variables B and C , and integer k such that:

1. $i \leq k < j$.
2. B is in X_{ik} .
3. C is in $X_{k+1,j}$.
4. $A \rightarrow BC$ is a production of G .

Finding such variables A requires us to compare at most n pairs of previously computed sets: $(X_{ii}, X_{i+1,j})$, $(X_{i,i+1}, X_{i+2,j})$, and so on, until $(X_{i,j-1}, X_{jj})$.

Construction of X_{ij}

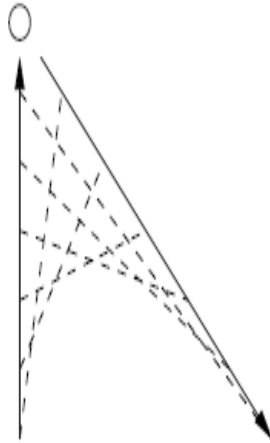


Figure 7.13: Computation of X_{ij} requires matching the column below with the diagonal to the right

Suppose we want to compute X_{14}

- The possible splits are $(X_{11}, X_{24}), (X_{12}, X_{34}), (X_{13}, X_{44})$
- The left component with position 1: $X_{11} X_{12} X_{13}$
- The right component with position 4: $X_{24} X_{34} X_{44}$

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12}	X_{23}	X_{34}	X_{45}	
X_{11}	X_{22}	X_{33}	X_{44}	X_{55}
a_1	a_2	a_3	a_4	a_5

Figure 7.12: The table constructed by the CYK algorithm

CYK Algorithm Example

- The CNF grammar:

$$\begin{array}{lcl} S & \rightarrow & AB \mid BC \\ A & \rightarrow & BA \mid a \\ B & \rightarrow & CC \mid b \\ C & \rightarrow & AB \mid a \end{array}$$

String to be checked ***baaba***

$\{S,A,C\}$				
- $\{S,A,C\}$				
- $\{B\}$	$\{B\}$			
$\{S,A\}$	$\{B\}$	$\{S,C\}$	$\{S,A\}$	
$\{B\}$	$\{A,C\}$	$\{A,C\}$	$\{B\}$	$\{A,C\}$
b	a	a	b	a

Figure 7.14: The table for string *baaba* constructed by the CYK algorithm

Undecidable CFL Problems

1. Is a given CFG G ambiguous?
2. Is a given CFL inherently ambiguous?
3. Is the intersection of two CFL's empty?
4. Are two CFL's the same?
5. Is a given CFL equal to Σ^* , where Σ is the alphabet of this language?

Turning Machine

- provides a mathematical model of a general-purpose computer.
- Introduced by Alan Turing in 1936.
- Provides an unbounded tape that serves as memory differing from FA and CFG.

Notation of Turing Machine

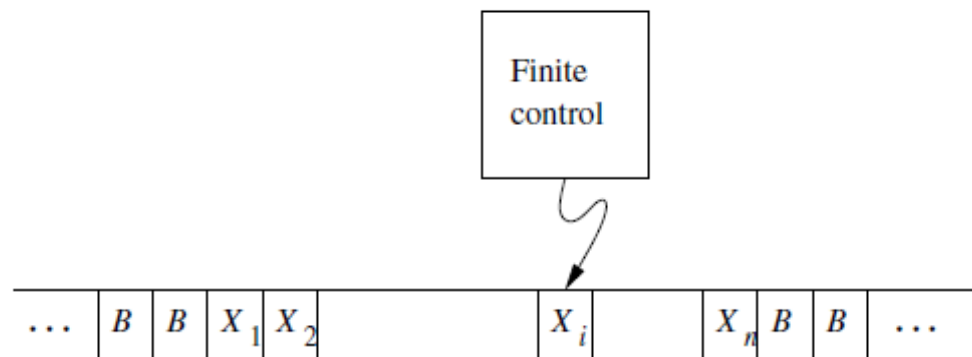


Figure 8.8: A Turing machine

A *move* of the Turing machine is a function of the state of the finite control and the tape symbol scanned. In one move, the Turing machine will:

1. Change state. The next state optionally may be the same as the current state.
2. Write a tape symbol in the cell scanned. This tape symbol replaces whatever symbol was in that cell. Optionally, the symbol written may be the same as the symbol currently there.

3. Move the tape head left or right. In our formalism we require a move, and do not allow the head to remain stationary. This restriction does not constrain what a Turing machine can compute, since any sequence of moves with a stationary head could be condensed, along with the next tape-head move, into a single state change, a new tape symbol, and a move left or right.

Formal Notation of a Turing Machine

The formal notation we shall use for a *Turing machine* (TM) is similar to that used for finite automata or PDA's. We describe a TM by the 7-tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

whose components have the following meanings:

Q : The finite set of *states* of the finite control.

Σ : The finite set of *input symbols*.

Γ : The complete set of *tape symbols*; Σ is always a subset of Γ .

δ : The *transition function*. The arguments of $\delta(q, X)$ are a state q and a tape symbol X . The value of $\delta(q, X)$, if it is defined, is a triple (p, Y, D) , where:

1. p is the next state, in Q .
2. Y is the symbol, in Γ , written in the cell being scanned, replacing whatever symbol was there.
3. D is a *direction*, either L or R , standing for “left” or “right,” respectively, and telling us the direction in which the head moves.

q_0 : The *start state*, a member of Q , in which the finite control is found initially.

B : The *blank symbol*. This symbol is in Γ but not in Σ ; i.e., it is not an input symbol. The blank appears initially in all but the finite number of initial cells that hold input symbols.

F : The set of *final* or *accepting* states, a subset of Q .

Instantaneous Description (ID)

string $X_1X_2\cdots X_{i-1}qX_iX_{i+1}\cdots X_n$ to represent an ID in which

1. q is the state of the Turing machine.
2. The tape head is scanning the i th symbol from the left.
3. $X_1X_2\cdots X_n$ is the portion of the tape between the leftmost and the rightmost nonblank. As an exception, if the head is to the left of the leftmost nonblank or to the right of the rightmost nonblank, then some prefix or suffix of $X_1X_2\cdots X_n$ will be blank, and i will be 1 or n , respectively.

Suppose $\delta(q, X_i) = (p, Y, L)$; i.e., the next move is leftward. Then

$$X_1X_2\cdots X_{i-1}qX_iX_{i+1}\cdots X_n \vdash_M X_1X_2\cdots X_{i-2}pX_{i-1}YX_{i+1}\cdots X_n$$

Notice how this move reflects the change to state p and the fact that the tape head is now positioned at cell $i - 1$. There are two important exceptions:

1. If $i = 1$, then M moves to the blank to the left of X_1 . In that case,

$$qX_1X_2\cdots X_n \vdash_M pBYX_2\cdots X_n$$

2. If $i = n$ and $Y = B$, then the symbol B written over X_n joins the infinite sequence of trailing blanks and does not appear in the next ID. Thus,

$$X_1X_2\cdots X_{n-1}qX_n \vdash_M X_1X_2\cdots X_{n-2}pX_n$$

Now, suppose $\delta(q, X_i) = (p, Y, R)$; i.e., the next move is rightward. Then

$$X_1X_2\cdots X_{i-1}qX_iX_{i+1}\cdots X_n \vdash_M X_1X_2\cdots X_{i-1}YpX_{i+1}\cdots X_n$$

Here, the move reflects the fact that the head has moved to cell $i + 1$. Again there are two important exceptions:

1. If $i = n$, then the $i + 1$ st cell holds a blank, and that cell was not part of the previous ID. Thus, we instead have

$$X_1X_2\cdots X_{n-1}qX_n \vdash_M X_1X_2\cdots X_{n-1}YpB$$

2. If $i = 1$ and $Y = B$, then the symbol B written over X_1 joins the infinite sequence of leading blanks and does not appear in the next ID. Thus,

$$qX_1X_2\cdots X_n \vdash_M pX_2\cdots X_n$$

Example-Construction of TM Accepting $\{0^n 1^n \mid n \geq 1\}$

In more detail, starting at the left end of the input, it enters a loop in which it changes a 0 to an X and moves to the right over whatever 0's and Y 's it sees, until it comes to a 1. It changes the 1 to a Y , and moves left, over Y 's and 0's, until it finds an X . At that point, it looks for a 0 immediately to the right, and if it finds one, changes it to X and repeats the process, changing a matching 1 to a Y .

$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \delta, q_0, B, \{q_4\})$$

State	Symbol				
	0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	—	—	—	—	—

Figure 8.9: A Turing machine to accept $\{0^n 1^n \mid n \geq 1\}$

$q_0 0 0 1 1 \vdash X q_1 0 1 1 \vdash X 0 q_1 1 1 \vdash X q_2 0 Y 1 \vdash q_2 X 0 Y 1 \vdash$
 $X q_0 0 Y 1 \vdash X X q_1 Y 1 \vdash X X Y q_1 1 \vdash X X q_2 Y Y \vdash X q_2 X Y Y \vdash$
 $X X q_0 Y Y \vdash X X Y q_3 Y \vdash X X Y Y q_3 B \vdash X X Y Y B q_4 B$

- Section 8.2 of Chapter 8
- Section 7.4 of Chapter 7